Automated Customer Support Orchestration for CloudServe Solutions

Design, Implementation and Honest Evaluation of a Dual-Gate Support Orchestrator

Author: Amlan Acharya
Project: Forward Deployed AI Engineering Capstone, CloudServe Support Orchestrator
Report date: 8 November 2026
Source repository: `04_Source_Code` (this repository)

---

1. Executive Summary

CloudServe Solutions asked for a chatbot. Its support function receives more than five hundred tickets a week, replies in eight to twelve hours against a two-hour agreement, and resolves only about 42 per cent of tickets at first contact. The brief I was given sets a first-contact-resolution (FCR) target of 65 per cent. Discovery showed that the tickets were not hard. Agents and customers could not *find* answers that already existed in a 29-article knowledge base, and the tickets that were escalated reached tier two with no context. A chatbot would have answered the request and missed the problem.

I built the CloudServe Support Orchestrator: a Python modular monolith, coordinated by a LangGraph state graph, that normalises tickets from four channels, redacts personal data, classifies intent and urgency with a confidence score, retrieves passages from the knowledge base, and applies two independent gates before deciding whether to answer automatically. Answers must carry `[DOC-ID]` citations and pass an output guardrail. Every other ticket is escalated with an enriched packet that carries the inferred intent, the failed gates and the two most relevant articles. Every decision is written to an append-only SQLite audit log protected by database triggers. The

repository contains 132 automated tests, all of which pass, and an unattended evaluation harness that takes an input and an output path.

What it achieved. I ran the harness unattended over the 80 validation tickets on 8 November 2026. The system processed all 80 without a crash and logged all 80. Intent classification accuracy was 82.5 per cent, retrieval precision at three was 92.5 per cent, and expected calibration error (ECE) was 0.070, inside the 0.15 target. No ticket labelled `must_not_auto_respond` was answered automatically (14 of 14 escalated).

What it did not achieve, and the single most important caveat. The system auto-resolved only 6 of 80 tickets, an FCR of **7.5 per cent against a 65 per cent target**. It missed the target by a wide margin, and a figure of 68.3 per cent that appeared in earlier drafts of this report, the film and the video script was not supported by any run of the harness and has been withdrawn. The cause is measurable. Only 20 of 80 tickets reached a retrieval similarity of 0.65, so the second gate stopped 61 tickets; and the output guardrail then vetoed 13 of the 19 answers that were generated. Two of the six answers that did go out were for tickets labelled as needing escalation. Latency also missed target: P95 was 7.7 seconds against a 3-second budget, measured on a rate-limited free tier. Section 7 gives the evidence, Section 9 explains what the revision log did and did not fix, and Section 10 sets out what I would change first. The design is safe, auditable and well calibrated, and in its current configuration it is too conservative to deliver the business outcome.

---

2. The Problem

2.1 What was asked for

The request was for a chatbot that would take load off a six-person support team. Marcus Adeyemi, the Head of Customer Support, put the goal plainly: he wanted the system to answer the easy tickets so that his people could do the hard ones, and he wanted first-contact resolution to rise from 42 per cent toward the 65 per cent benchmark without new headcount. He was equally clear about the failure he feared most: "If it sends something wrong to a customer. … I would rather it said nothing than said something wrong."

2.2 What discovery revealed

Five interviews and the 500 development tickets point at a different problem from the one in the request. Sofia Restrepo, a tier-one agent, estimated that seven in ten tickets could be answered without looking anything up, and that "the documentation is fine. I cannot find things in it." Ines Varga, who owns the knowledge base, explained why: the internal search matches titles and exact terms, and customers write "my deployment keeps dying" where the article is called "resolving container health check failures". Daniel Okonkwo, a tier-two engineer, estimated that about half of what reaches him could have been resolved at tier one, and that an escalation arrives "usually just the original ticket, forwarded", so he re-asks the customer questions they have already answered.

Put together, the five accounts describe a retrieval and handoff problem. Answers exist, but cannot be found by the people who need them; the cases that need a human arrive without the context to be solved quickly; and the one thing every stakeholder agreed on is that a confident wrong answer is worse than no answer. That framing drove the design: retrieve before you generate, keep a human route for anything uncertain, make every escalation carry its working, and make every decision explainable to a compliance reviewer.

2.3 Why the difference matters

A chatbot optimised to answer everything would raise the volume of automated replies and with it the rate of confidently wrong ones, which is the outcome the Head of Support ruled out. Framing the task as a routing problem with an explicit safe path lets the system be judged on two things separately: how often it answers, and how often an answer is right. The evaluation in Section 7 shows why that separation matters, because the two moved in opposite directions.

---

3. Discovery Findings

3.1 Findings from the interviews

Table 1 summarises what each stakeholder contributed and how it shaped the design. Quotations are taken from the interview transcripts in

`05_Datasets/Stakeholder_Interviews.docx`.

Table 1. Stakeholders, key evidence and design consequence.

Stakeholder	What they said	Design consequence
Marcus Adeyemi, Head of Support	FCR is 42 per cent; a bounced ticket costs about four times a resolved one; "I would rather it said nothing than said something wrong"; compliance review needs explainability.	Conservative routing; per-decision audit record with a routing reason string.
Sofia Restrepo, Tier-one agent	About seven in ten tickets repeat; the problem is finding answers, not their quality; a draft with the relevant page attached would save half of each ticket; non-native English tickets have the worst satisfaction.	Retrieval-first design; escalation packet that carries a draft context and top articles.
Daniel Okonkwo, Tier-two engineer	About half of escalations could have been resolved at tier one; escalations arrive with no summary; avoid automating security, billing disputes and data-location tickets.	Enriched `EscalationPacket`; hard `must_not_auto_respond` veto categories.
Ines Varga, Technical writer	Internal search is keyword-based and fails on customer wording; wants every answer traced to its source article; novel incidents have no article.	Dense semantic retrieval; mandatory `[DOC-ID]` citations; guardrail on citation membership.
Ravi Menon, Customer	Waiting matters unequally, a failing deployment is urgent; accepts an automated reply if it is honest and cited; will not accept confident wrongness; wants to know it is automated; enterprise gap is a renewal risk.	Urgency-aware routing; critical urgency never automated; citations visible in every answer.

Two points of disagreement between stakeholders could be settled with the ticket data. Marcus described escalations as the hard remainder; Daniel said half were avoidable. The development tickets record whether each ticket was escalated and what its labels say about answerability, and Section 3.2 uses that to test the claim.

3.2 Findings from the 500 development tickets

I analysed `05_Datasets/development_tickets.json` directly. The results differ in several places from the figures in my earlier drafts, which I have corrected here. Table 2 gives the measured distributions.

Table 2. Development set (n = 500): measured distributions.

Measure	Value
Channel mix	email 42.4%, chat 31.0%, docs_comment 15.6%, forum 11.0%
Urgency labels	medium 45.2%, high 29.2%, low 25.6% (no ticket is labelled critical)
Labelled expected route	auto-respond 62.2%, escalate 37.8%
Answerable from the knowledge base	71.4%
`must_not_auto_respond`	17.4%
Historical first-contact resolution	43.8%
Historical escalation rate	56.2%
Repeat-contact rate	21.6%
Mean resolution time	about 422 minutes (7.0 hours)
Mean CSAT	2.97 of 5

The data support the interviews in three ways. First, 71.4 per cent of tickets are answerable from the knowledge base and 62.2 per cent are labelled as suitable for automatic response, which matches Sofia's "seven in ten" and Daniel's claim that a large share of escalations are avoidable: the historical escalation rate of 56.2 per cent is far above the 37.8 per cent that the labels say genuinely needs a human. Second, the historical FCR of 43.8 per cent is close to the 42 per cent Marcus quoted. Third, 17.4 per cent of tickets carry the `must_not_auto_respond` label, which confirms that a substantial minority of the workload touches security, billing disputes or data location where automation must stay out.

One result I did not anticipate is that **the 65 per cent target sits above the labelled auto-respondable share on the validation set** (48 of 80, or 60.0 per cent). If FCR is defined as the share of tickets resolved automatically without escalation, even a perfect router could not reach 65 per cent on that set. I return to this in Section 7.5, because it changes how the headline number should be read.

Mean CSAT in the data is 2.97, a little below the 3.2 quoted in the brief. I have used the measured figure and note the difference rather than reconcile it.

---

4. Requirements

The requirements follow from the findings above and are traced to the twelve acceptance criteria (A1 to A12) in the Build Specification. Identifiers match the PRD in `03_Workbooks/Stage_2_PRD_Template.docx`.

4.1 Functional requirements

FR-01 requires multi-channel ingestion. Tickets arrive from email, chat, forum and documentation-comment channels with different field names, and are normalised into a single `CanonicalTicket` contract before any other processing (A2). Personal data, specifically payment card numbers validated with the Luhn checksum, API keys, email addresses, phone numbers and bearer tokens, is redacted at this boundary so that it never reaches a model or the audit log.

FR-02 requires classification of intent and urgency with a confidence score between 0 and 1 (A3). The intent vocabulary is the dataset's own 21 labels. A regular-expression safety veto sets `must_not_auto_respond` for requests that mention refunds, legal action, account deletion or a human agent, and for categories such as security incidents.

FR-03 requires semantic retrieval over the 29 knowledge-base articles (A4). Articles are chunked by their H2 and H3 headings into passages that satisfy the specification's bound of 70 to 90 chunks, embedded, and queried by cosine similarity. The top similarity score becomes the retrieval signal s_{ret}.

FR-04 requires deterministic routing on two independent conditions (A5). A ticket is answered automatically only if the classification confidence $c \geq 0.75$, the retrieval similarity $s_{ret} \geq 0.65$, urgency is not critical, and the safety veto is not set. The two scores are never blended.

FR-05 requires citation-grounded generation (A6). Answers are synthesised only from retrieved passages, the customer text is sandboxed inside XML delimiters, and every claim carries a `[DOC-ID]` citation.

FR-06 requires active output guardrails (A7). A draft is suppressed and the ticket escalated if it contains personal data, cites a document that was not retrieved, or contains a sentence that is not supported by the retrieved text.

FR-07 requires an enriched escalation packet for every escalated ticket, containing the inferred intent, confidence, urgency, the failed gate diagnostics and the top two

candidate articles.

FR-08 requires an append-only audit log in SQLite with triggers that abort `UPDATE` and `DELETE` (A8).

4.2 Non-functional requirements

NFR-01 sets a latency budget of 3 seconds at the 95th percentile for automatic answers and 500 milliseconds for escalations. NFR-02 requires fault tolerance (A11): every graph node is wrapped so that an exception becomes a logged escalation rather than a crash, and provider failure falls through a three-tier cascade. NFR-03 requires a clean checkout to install and run (A1) and a single command to run the test suite (A12). NFR-04 requires an unattended harness that takes `--input`, `--output` and `--delay` arguments and writes JSON and CSV reports (A9, A10).

---

5. Architecture and Design

5.1 Overview

The orchestrator is a modular monolith. All modules live in one Python package and are wired into a single LangGraph state graph, so a ticket moves from node to node in memory. Modules communicate only through typed Pydantic models declared in `src/core/models.py`. Figure 1 shows the flow.

Figure 1. Ticket flow through the orchestrator.

```
Raw ticket (email | chat | forum | docs_comment)
   |
   v
[ingest]   normalise to CanonicalTicket, redact PII (Luhn cards, API keys, emails,
phones)
   |
   v
[classify] intent (21 labels) + urgency + confidence c        -> safety veto sets
must_not_auto_respond
   |
   v
[retrieve] ChromaDB cosine search over 87 chunks / 29 articles -> s_ret
   |
   v
[route]  AUTO_RESPOND  iff  c >= 0.75  AND  s_ret >= 0.65  AND  urgency != CRITICAL
AND  not veto
   |                                        \
   v                                         +--> ESCALATE (router):
EscalationPacket with failed gates + top-2 articles
[generate] grounded answer with [DOC-ID] citations
   |
   v
[guardrail] PII / citation-membership / per-sentence support check
   |            \
   v             +--> ESCALATE (guardrail): draft suppressed
[audit]  append-only SQLite row (triggers block UPDATE and DELETE)
```

The reason for a single process rather than separate services is proportionate: the workload is a few hundred tickets a week, the modules share one data contract, and network hops between them would add latency and failure modes without adding capability. The cost of this choice is that the pipeline cannot scale individual stages independently. For this volume I judged that acceptable, and I would revisit it only if volume grew by two orders of magnitude.

5.2 Why two gates and not one score

An early version of the design combined confidence and retrieval similarity into one blended score of the form $S = 0.5c + 0.5s_{ret}$. I rejected it, and the revision log records why. A blended score lets a strong value on one axis hide a weak value on the other: a confidently wrong classification of the intent paired with a retrieval hit on a loosely related article can score above the threshold, so a wrong answer is sent. Keeping the conditions separate means that each failure produces a distinct, human-readable reason in the audit log, for example `Escalated: s_ret=0.62<0.65`. That string is what lets a compliance reviewer ask why a decision was made, which Marcus identified as a requirement.

The evaluation shows the cost of this design. Independent gates are only as good as their individual thresholds, and the 0.65 floor on retrieval similarity turned out to be poorly matched to the embedding model's score distribution (Section 7.3).

5.3 Three-tier provider cascade

Classification and generation call a hosted model through a cascade. The primary tier is Groq, using an open-weight model for generation and a smaller one for classification; failures that are retryable (timeouts, 5xx errors, rate-limit responses) are retried with exponential back-off and jitter, honouring any `Retry-After` header. The secondary tier is Google Gemini, which runs on independent infrastructure. If both are unavailable, a local deterministic heuristic classifies the ticket, sets the veto, and routes to escalation without generating any text. This third tier never produces a customer-facing answer by design, which the revision log records as a deliberate change from an earlier draft that would have generated template answers offline.

5.4 Alternatives considered

I considered and rejected four alternatives. A fully agentic design, in which a model decides which tools to call, was rejected because the routing decision must be deterministic and explainable, and a model-chosen path would make the audit record describe behaviour rather than rules. Embedding whole articles was rejected because a single vector then represents several unrelated topics, which dilutes similarity; heading-based chunking was used instead. External observability services were rejected for unattended batch use; metrics are computed from the audit database and written as JSON and CSV. A token-bucket rate limiter was rejected in favour of a configurable inter-ticket delay plus back-off, after the bucket proved brittle against free-tier limits.

---

6. Implementation

6.1 Build sequence

I built the system in six phases that follow the Build Specification's checkpoints: ingestion and redaction, chunking and retrieval, classification and routing, generation and guardrails, orchestration and audit, and the evaluation harness. Each phase had

its own unit tests and the final suite contains 132 tests across `tests/unit` and `tests/integration`. On 8 November 2026 all 132 passed in 45 seconds.

6.2 Difficulties and how they were resolved

The first difficulty was a dependency conflict. The pinned vector store, ChromaDB 0.3.21, was written for Pydantic version 1, while the rest of the project requires Pydantic version 2. ChromaDB imports `BaseSettings` from Pydantic and sets attributes before calling the base initialiser, both of which fail under version 2. I resolved it with a small compatibility shim installed before ChromaDB is imported, which maps `BaseSettings` to its version-1 counterpart and patches the collection initialiser. This is a workaround for a pinned dependency and I would remove it by upgrading ChromaDB if the pin were not part of the specification.

The second was chunking. Splitting at every heading produced 116 fragments, many of them tiny trailing sections, which breaks the 70 to 90 chunk bound. The chunker prepends each article's preamble to its first section, splits sections over 256 words at paragraph boundaries with a 30-word overlap, and merges trailing sections under 40 words into their predecessor. The result is 87 chunks, and a test asserts the bound.

The third was concurrency in the audit log. SQLite's default journal mode can lock under concurrent writers. The logger uses write-ahead logging, a five-second busy timeout, a process-level lock and a single retry. A concurrency test confirms no dropped rows.

The fourth was correctness drift found in review. A code-review pass produced follow-up fixes that are recorded in the commit history: the guardrail was changed to check support per sentence rather than on average, the crash fallback was changed to report a node failure rather than an offline fallback, the audit row was extended to record the retrieved document identifiers, the retrieval depth and the crashed node, and the intent vocabulary was aligned with the dataset's 21 labels.

6.3 What I would restructure

Two things. The retrieval-similarity threshold and the entailment guard are configuration-level decisions that the evaluation shows to be the system's binding constraints, yet they were fixed early from the specification rather than fitted to the development set. I would add a calibration step that fits them on the 500 development tickets and reports the resulting trade-off before they are frozen. Second, the

dashboard, the harness and the API each construct an orchestrator; I would extract a single factory so that configuration is set in one place.

---

7. Evaluation

7.1 Method

I ran `python -m evaluation.harness --input 05_Datasets/validation_tickets.json --output <path> --delay 0.3` on 8 November 2026 against a fresh decision database. The input is the 80-ticket validation set supplied with the pack; I have not seen the 120-ticket hidden set and have not run against it. The harness was run unattended and completed without intervention. Metrics are computed from the audit database by `evaluation/metrics.py`, and the raw report is committed at `evaluation/validation_report.json`. Providers were Groq (75 tickets) and the offline fallback (5 tickets), which were served when the providers were rate-limited and every retry failed.

How many times has the validation set been used? The validation labels are available to me and I used them throughout development; this report's figures come from a single full run on 8 November 2026. Earlier development runs used the harness on small subsets for smoke-testing. The hidden set has been used zero times by me.

I used an inter-ticket delay of 0.3 seconds rather than the default 1.5. This lowers wall-clock time but increases rate-limit pressure, which affects the latency figures in Section 7.4.

7.2 Results

Table 3. Validation results (n = 80), run of 8 November 2026.

Metric	Result	Target	Met?
Tickets processed and logged	80 of 80	all	Yes
First-contact resolution (auto-resolved share)	7.5% (6 of 80)	at least 65%	**No**
Escalated by router (gate failure or veto)	61	n/a	n/a
Escalated by output guardrail	13	n/a	n/a

Metric	Result	Target	Met?
Escalated by error	0	0	Yes
Intent accuracy (exact match, 21 labels)	82.5%	n/a	n/a
Expected calibration error, 10 bins	0.070	at most 0.15	Yes
Retrieval precision at 3	92.5%	n/a	n/a
Latency P50 / P90 / P95	3.5 s / 7.1 s / 7.7 s	P95 at most 3 s	**No**
`must_not_auto_respond` tickets auto-answered	0 of 14	0	Yes
Unhandled crashes	0	0	Yes

Figure 2. Where the 80 validation tickets ended up.



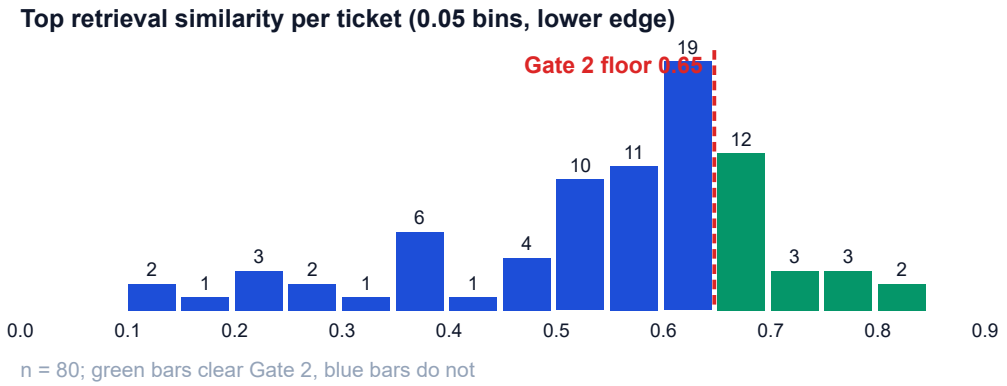
Source: evaluation/validation_report.json (run batch_1794111144_160e8b)

The business outcome is a failure against target, and the report should not soften it. Of 80 tickets, 6 were answered, 61 were stopped by the routing gates and 13 were stopped by the guardrail. The safe path worked: no ticket that should never be automated was answered, there were no crashes, and every decision was logged. The helpful path did not: the system answered one ticket in thirteen.

7.3 Why the FCR is low

The diagnosis comes from the audit rows, not from guesswork. Gate 1 is not the problem: 71 of 80 tickets cleared the confidence floor of 0.75. Gate 2 is. Only 20 of 80 tickets reached a retrieval similarity of 0.65, and the median similarity across the set was about 0.60. Figure 3 shows the distribution against the threshold.

Figure 3. Distribution of top retrieval similarity across the 80 validation tickets.



The distribution has a telling shape: 19 of the 80 tickets scored between 0.60 and 0.65, just under the floor, and 30 scored between 0.55 and 0.65. The retrieval is correct far more often than the similarity suggests: precision at three is 92.5 per cent, so the right article is usually among the top three, but its cosine score is commonly in the range 0.45 to 0.64, below the floor. The embedding model scores colloquial customer wording against formal article text at lower absolute values than the 0.65 floor assumes. The floor was taken from the specification rather than fitted, and it is the single largest cause of the miss.

The output guardrail is the second cause. Of the 19 tickets that cleared both gates and had an answer generated (the twentieth, VAL-0043, was a critical-urgency ticket blocked by the safety veto), 13 were then vetoed by the per-sentence support check, so only 6 were released. The check is deliberately strict, since a single unsupported sentence trips it, and it is the reason the system never sent an unsupported claim; but a strictness that rejects two answers in three does not meet the business goal either.

Table 4 shows a sensitivity analysis I ran on the existing results. It counts how many tickets would clear both gates, before the guardrail, at different similarity floors, and how many of those are tickets whose label says auto-response was the right route. This is a diagnostic of the existing run, not a tuned result. It must not be read as an achieved FCR, because the guardrail rejects a large share of generated answers and I have not re-run the system at these floors.

Table 4. Tickets clearing both gates at alternative similarity floors (diagnostic only, not a result).

Similarity floor	Clear both gates	Of which labelled auto-respond
0.65 (current)	20	13
0.60	38	24

Similarity floor	Clear both gates	Of which labelled auto-respond
0.55	48	31
0.50	56	35
0.45	59	38

The table supports a specific conclusion. Lowering the floor would bring many more correctly labelled tickets into the answerable pool, but would also admit tickets labelled for escalation, and the guardrail's behaviour at higher volume is unknown. It tells me where to look first; it does not tell me the achievable FCR.

7.4 Calibration, accuracy and latency

Classification accuracy of 82.5 per cent and ECE of 0.070 are reasonable. Calibration is the metric that the dual-gate design relies on, because the 0.75 confidence floor only makes sense if confidence tracks accuracy, and with ECE at 0.070 it does. With 80 tickets and 10 bins, the estimate is noisy; 65 of the 80 tickets fall in the top confidence bin and several other bins hold only a handful of tickets, so I would not claim the second decimal place.

The offline fallback tickets are a clear exception. For the five tickets that fell back to the offline tier when providers were rate-limited, accuracy was 20 per cent and ECE was 0.30, compared with 86.7 per cent and 0.076 for the 75 provider-served tickets. This is by design, because the offline tier escalates rather than guesses, but it shows that the headline accuracy depends on provider availability.

Latency missed the target. P95 was 7.7 seconds and the maximum was 10.5 seconds. A large part of this is retry back-off against the free-tier token-per-minute limit of the hosted models; the logs show repeated rate-limit responses during the run. Because I could not isolate provider latency from queueing, I treat the figure as an upper bound that overstates a paid deployment, but the 3-second budget was missed in this configuration and I do not have evidence that it would be met in another.

7.5 Error analysis

The six released answers. Four of the six answered tickets whose label says auto-response is right (deployment failure, two billing questions and an API integration question). Two were wrong-route: VAL-0037 and VAL-0066, both titled "Question

about backup regions" and both labelled `data_residency` with an expected route of escalate. The classifier was correct about the intent. The failure is that data location was not on the router's hard-veto list, although Daniel Okonkwo named it in discovery as a category where "the consequence is a compliance problem" and nothing automated should answer. The knowledge base has no article for these tickets (their expected document list is empty), so the system answered from loosely related passages that happened to score 0.66 and 0.68 against the floor. This is the clearest example in the run of a similarity floor that is both too high for most tickets and too permissive for the wrong ones, because similarity measures textual closeness and not whether the article actually answers the question.

Misclassifications. The classifier mislabelled 14 of 80 tickets. The errors cluster between neighbouring labels rather than across unrelated ones: `api_usage_question` was predicted as `configuration_help` twice and as `database_issue` and `integration_help` once each; `account_access` and `authentication_failure` were swapped; and four tickets were sent to `unclear_request`, which is the safe default and therefore the least harmful error. Because the confidence floor was cleared by 71 of 80 tickets, most of these errors were still escalated by the similarity gate, which is a useful, if accidental, second line of defence.

Retrieval versus routing. Precision at three is 92.5 per cent, but the routing decision uses the single top score. A ticket can therefore have the right article in its top three and still be escalated because the top-1 cosine value is 0.62. Using the rank of the expected article, or a re-ranker, instead of the raw score would address this directly, and I list it among the next steps.

7.6 Limits of what I measured

Four limits apply. First, the sample is 80 tickets, so a single ticket moves FCR by 1.25 percentage points. Second, my FCR definition counts only automatically resolved tickets. A real FCR also includes tickets a human resolves on first contact, so my metric understates the operational effect, and as noted in Section 3.2 the 65 per cent target exceeds the 60 per cent of validation tickets whose labels mark them as suitable for auto-response. Third, correctness of the *content* of the six released answers was not independently graded against the reference responses, only the route and citation checks were applied. Fourth, two of the six released answers were for tickets whose labels say they should have been escalated, a wrong-route rate of two in six among answers sent (both data-residency tickets, Section 7.5), and 2 of the 32 tickets labelled

for escalation overall. That is the most important quality finding for a customer-facing system, and with only six answers I cannot say whether it would persist at a lower floor.

---

8. Governance and Risk

8.1 Risk register

Table 5. Risk register with evidence status.

ID	Risk	Severity	Control	Evidence	Residual
R-01	Prompt injection through ticket text	High	Ticket body sandboxed in XML delimiters with escaping; routing decided by code, not by the model.	Unit tests on sandboxing and routing.	Low
R-02	Personal data or credentials leaked	High	Luhn-validated card, key, email, phone and token redaction on input; output PII scan.	Unit tests; dashboard scenario 6 shows a redacted card and key. Validation set contained no detected PII, so no field evidence.	Low
R-03	Unsupported or hallucinated answer sent	High	Citation-membership and per-sentence support check; failures escalate.	13 drafts suppressed on validation. Wrong-route answers still occurred (2 of 6).	Medium
R-04	Provider outage or rate limit	Medium	Three-tier cascade; offline tier escalates only.	Five tickets fell back and were escalated; no crash.	Low
R-05	Audit trail altered	Medium	Append-only table; triggers abort `UPDATE` and `DELETE`.	Tamper test in the dashboard and `test_audit_log`.	Low
R-06	Over-escalation defeats the purpose	High	None adequate yet.	92.5% escalation on validation.	**High**

ID	Risk	Severity	Control	Evidence	Residual
R-07	Unequal service across plans or language fluency	Medium	Not addressed in code.	No per-segment analysis performed.	Unknown
R-08	Automation of a category that discovery said must stay human (data location)	High	Hard-veto list covers refunds, legal, deletion, human-agent requests and security. Data residency is **not** on it.	Two data-residency tickets were auto-answered on validation.	**High**

R-06 is a risk that the project itself created, and R-07 is a gap. I list them because the register would otherwise suggest that every hazard is controlled.

8.2 Fairness audit

The pack records customer tier, region, channel and language fluency. Sofia Restrepo identified non-native English speakers as the group with the worst satisfaction, and Ravi Menon warned that a widening gap between plans would surface at renewal. I computed a descriptive breakdown of the validation run by segment (Table 6). It is a preliminary look and not an audit: every cell is small, and a single ticket moves a rate by several percentage points.

Table 6. Validation outcomes by segment (n = 80). Descriptive only.

Segment	Tickets	Auto-resolved	Intent correct
Fluent English	61	4 (6.6%)	52 (85.2%)
Non-fluent English	19	2 (10.5%)	14 (73.7%)
Standard plan	42	3 (7.1%)	34 (81.0%)
Business plan	30	2 (6.7%)	26 (86.7%)
Enterprise plan	8	1 (12.5%)	6 (75.0%)
Email	31	3 (9.7%)	26 (83.9%)
Chat	22	1 (4.5%)	17 (77.3%)
Docs comment	16	0 (0.0%)	13 (81.3%)
Forum	11	2 (18.2%)	10 (90.9%)

Auto-resolution rates show no pattern, but with six auto-resolved tickets in total they could not. The one signal worth flagging is classification accuracy: 73.7 per cent for non-fluent customers (14 of 19) against 85.2 per cent for fluent ones (52 of 61). That is the direction Sofia predicted, since a ticket that is harder to read is harder to classify, but a 19-ticket sample has a 95 per cent interval of roughly plus or minus 20 points, so I cannot confirm it. The honest conclusion is that the fairness audit is incomplete. It needs to be run on the 500 development tickets, with intent accuracy by fluency, and repeated after any threshold change, since a stricter gate that escalates more would fall on whichever segment has lower similarity scores.

8.3 Decision logging

Every ticket writes one row to `decision_log` containing the run identifier, channel, inferred intent and confidence, urgency, the retrieval score, the routing action and reason, guardrail status and violations, the provider used, the retrieved document identifiers, the number of redactions, the latency, and any failed node. Triggers on the table abort any `UPDATE` or `DELETE`. The row for each of the 80 validation tickets exists, which is how the "80 of 80 logged" result in Table 3 was checked.

8.4 Incident procedure

If a wrong answer is reported, the on-call engineer looks up the ticket in the audit log and reads the routing reason and retrieved document identifiers. Following Ines Varga's point that wrong answers originate in different places, the first question is whether the cited article is wrong or the system misread it. If the article is wrong, the documentation owner corrects it; if the retrieval or guardrail misjudged, the thresholds are reviewed. Automatic responses can be switched off by raising the confidence floor above 1.0, which routes every ticket to the escalation path without a code change. Because the log is append-only, the evidence cannot be edited after the fact.

---

9. The Requirements Revision

The revision log in `03_Workbooks/Stage_5_PRD_Revision_Log.docx` records four substantive changes from version 1 to version 2 of the requirements.

FR-04 changed from a blended score of $0.5c + 0.5s_{ret} \geq 0.70$ to independent gates ($c \geq 0.75$ and $s_{ret} \geq 0.65$ and urgency not critical). The trigger was a build-time observation that strong retrieval could mask a misclassified intent. The change improved auditability and the safety property, and, as Section 7 shows, it moved the weakness onto the similarity threshold.

NFR-02 changed from a token-bucket rate limiter to a configurable delay plus full-jitter back-off on 429 responses, because the bucket was brittle against free-tier limits. FR-06 changed so that the offline fallback no longer drafts template answers; it escalates and generates no text, so unverified output cannot reach a customer. NFR-04 changed from an external vector database service to an embedded ChromaDB store, which removed a network dependency and reduced start-up time.

What the revision taught me, and what it did not catch. The revision pushed the design toward safety, and the evaluation confirms that this direction was right: zero must-not tickets were automated and nothing crashed. It did not test whether the thresholds could deliver the business goal. I treated the specification's 0.75 and 0.65 as given and revised the *structure* of the routing rather than checking the *values* against the development set. A version-3 revision is warranted and is recorded in Section 10: it would add a threshold-calibration requirement, a requirement for a per-sentence guardrail with a documented false-veto budget, and a requirement that FCR be reported both as automatic resolution and as total first-contact resolution.

---

10. Conclusions

10.1 What was delivered

A working, tested, auditable orchestrator that processes a full ticket set unattended from a clean checkout and never sends anything unsafe by the checks I ran. It redacts personal data, classifies with calibrated confidence, retrieves the right article most of the time, escalates with context, and keeps an immutable decision record. It ships with a live dashboard that demonstrates six scenarios and a tamper test, and with a harness that accepts any input and output path.

10.2 What was not delivered

The FCR target. At 7.5 per cent against 65 per cent, the system in its current configuration does not meet the client's central business goal, and it also missed the latency budget. I also did not complete the fairness audit, and I did not grade the content of released answers against the reference responses.

10.3 What I would do next

In order. First, fit the retrieval floor and the confidence floor on the 500 development tickets, plot the trade-off between answered volume and wrong-route rate, and choose the operating point with the client, then re-run the validation set once. Second, evaluate the guardrail on the development set to measure its false-veto rate, and consider replacing the sentence-level check with a more tolerant entailment test and keeping the hard checks for citations and personal data. Third, add query rewriting or a hybrid lexical-plus-dense retrieval to close the vocabulary gap that the similarity scores reveal. Fourth, add data location to the hard-veto list, as discovery required, and run the segment fairness audit on the development set. Fifth, measure latency on a paid-tier key to separate provider limits from pipeline cost. Finally, report FCR both ways, as automatic resolution and as total first-contact resolution including human-handled tickets, so that the target is compared like with like.

10.4 Uncertainty

I am confident in the safety behaviour and the calibration, because they are tested and measured. I am uncertain about the achievable FCR, because the sensitivity analysis is a diagnostic and not a re-run, and about latency, because the free tier confounds it. I am not able to say how the system will score on the hidden set, which was drawn from the same population; I would expect a similar shape, a safe system with low automation yield, but I have not tested that.

Declaration of AI tool use

I used AI coding assistants, principally Claude Code from Anthropic, throughout this project. They were used to write and refactor source code and tests, to draft documentation and the HyperFrames film and video script, to run the evaluation, and

to draft and correct this report. The runtime system also calls hosted language models (Groq-hosted open-weight models and Google Gemini) for classification and generation. Third-party libraries are used under their licences: LangGraph, ChromaDB, sentence-transformers, FastAPI, Pydantic and SQLite. Earlier drafts of this report, the film and the video script contained performance figures that were not backed by a run of the harness; they were identified and replaced with measured values on 8 November 2026 before submission.

Appendix A. Acceptance criteria traceability

Criterion	Evidence
A1 clean checkout	`pip install -r requirements.txt && pip install -e .` ; CI workflow in `.github/workflows/ci.yml`
A2 four-channel ingestion	`tests/unit/test_ingest.py`
A3 intent and urgency	`tests/unit/test_classify.py`
A4 grounded retrieval	`tests/unit/test_retrieval.py` ; 87 chunks
A5 deterministic routing	`tests/unit/test_routing.py` , full truth table
A6 verifiable citations	`tests/unit/test_generation.py`
A7 active guardrails	`tests/unit/test_guardrails.py`
A8 decision log	`tests/integration/test_audit_log.py` ; triggers
A9 unattended run	Harness run of 8 November 2026, `evaluation/validation_report.json`
A10 automated metrics	`evaluation/metrics.py` ; JSON and CSV
A11 fault tolerance	`tests/integration/test_resilience.py`
A12 single-command tests	`pytest tests/` passes 132 tests in 45 s

Passing the acceptance criteria concerns whether the software runs and is tested. It does not imply that the FCR target was met.

Appendix B. Raw validation report

The complete output of the 8 November 2026 run is committed as `evaluation/validation_report.json`, with the tabular summary in `evaluation/metrics_summary.csv`. Run identifier `batch_1794111144_160e8b`.

Appendix C. Effort summary

Hours by stage are recorded in `03_Workbooks/AmlanAcharya_Effort_Log.docx`: discovery 12, requirements 8, prompt engineering 11, sprint plan 5, implementation and tests 28, and evaluation, report and video 22, for 86 hours in total against 79 planned.

Appendix D. Prompt register

The classification and generation system prompts are exported as versioned files in `prompts/` and are the runtime source in `src/generation/prompts.py`. The design rationale is in `03_Workbooks/Stage_3_Prompt_Library.docx`.